

澳門理工大學
Universidade Politécnica de Macau
Macao Polytechnic University

Faculty of Applied Sciences

Bachelor of Science in Computing

COMP413 Enterprise System and Application Development

Project Report

Academic Year 2025/26

Library management system

Student ID:	P2111671
Student name:	Rui Wang
Class:	411(A)
Submission date:	2025/12/07

Content

1. Requirement Analysis 1

1.1 Business Need 1

1.2 Core Functional Requirements 1

1.3 Non-Functional Requirements 3

1.3.1 Security 3
1.3.2 Usability 3
1.3.3 Reliability and Data Integrity 4
1.3.4 Scalability 4

2. System Architecture Design 5

2.1 Client Tier 5

2.2 Web Tier 6

2.3 Business logic Tier 6

2.4 Enterprise Information System (EIS) Tier 6

3. Component Design 7

3.1 Web Tier Components 7

3.2 Business Logic Tier Components 9

3.3 Enterprise Information System (EIS) Tier Components 10

3.4 UML Class Diagram 11

4. Database Design 12

4.1 ER Diagram 12

4.2 Schema Design: 13

5. Interface Design 14

5.1 Site map 14

5.2 HCI principle 14

5.2.1 Strive for consistency 14
5.2.2 Offer informative feedback 15
5.2.3 Prevent errors 16

5.2.4 Permit easy reversal of actions 17

5.3 Sequence Diagrams 18

5.3.1 Sequence Diagram 1 — Borrower Submits Borrow Request 18
5.5.2 Sequence Diagram 2 Introduction — Librarian Approves or Rejects Borrow Request 19
5.5.3 Sequence Diagram 3 Introduction — Borrower Return Request, Librarian Confirms Return, and Auto-Overdue Handling 20

5.4 Interface disclosure 22

5.4.1. General Public 22
5.4.2 Administrators 25
5.4.3 Advanced feature: 30

6. Testing 34

6.1 Unit Testing 34

6.2 Integration Testing 34

6.3 Edge Case Handling 35

6.4 Bug Resolution Process 36

6.5 Conclusion 36

1. Requirement Analysis

1.1 Business Need

This library management system is designed and open to give resources to the public and potential users and borrowers to borrow books and provide an intuitive interface for librarians to manage transactions between books and borrowers to ensure a fluent workflow. Also, the system administrators are entitled to add and manage the librarians.

1.2 Core Functional Requirements

The library management system has two main kinds of users: **borrowers** and **librarians**. Besides, a **system administrator** has been added to manage the librarian's account. Borrowers access the user portal to browse books and manage their borrowing activities, while librarians manage the book catalog, book copies, and borrowing/return requests in the administration portal. The user portal provides four main functions for borrowers: book search, book details, borrowing cart (loan requests), and view borrowing records. The interface should be designed to be easy to use and intuitive for library members.

A1: Borrowers may register a new account. They have to provide their first name, last name, date of birth (DOB), gender, email address, and password during registration.

A2: Book search and book detail information are accessible to borrowers before login. After logging in, borrowers can apply to borrow books and manage borrowing records. Borrowing records and other borrower information are stored on the server side and persisted across user sessions.

A3: Borrowers can view their personal information (name, email, etc.) after login and are allowed to change their password when needed.

A4: Borrowers may browse numerous books in a list. The book list presents essential information for each book, including the title, author, and category.

A5: Borrowers may browse and search numerous books by title, author, category, or other attributes.

A6: Borrowers can view book detail information. The detail page displays additional attributes beyond the title, author, and category, including at least the ISBN, publisher, and publication year, a description, a cover image, and the number of available copies.

A7: Borrowers can request to borrow a book from its detail page. If at least one physical copy is available, the system will allow the borrower to submit a borrowing request, which will then be recorded in the system.

A8: Borrowers can list the books they have borrowed or requested to borrow in borrowing records. For each borrowed item, the system displays the book title, the borrow date, the due date, and the borrowing status.

A9: Borrowers need to manually request a book return on their own Borrowing record page.

The administration portal is used by librarians to manage the book catalog, book copies, borrowers' information, and borrowing/return processing.

A10: Librarians can view their personal information (name, email, etc.) after login and are allowed to change their password when needed.

A11: Librarians are able to perform full CRUD operations on books. When adding a new book, the librarian must provide the title, author, ISBN, publisher, publication year, category, and description, with an optional cover image upload. All these attributes may be modified later if necessary.

A12: Librarians can browse the books in the book catalog, and search books by title (and optionally other book attributes such as author, ISBN, or category).

A13: Librarians can disable a book to temporarily hide it from borrowers in the user portal. The librarian can later enable the book again.

A14: Librarians can view borrowers' basic information (name, email) to facilitate contact.

A15: Librarians can search for specific books using the title, ISBN, or author.

A16: The librarian can browse and search the list of all physical book copies. Each copy entry shows the Copy ID, the associated Book Title, and the copy status.

A17: The librarian can add a new physical book copy in the system. The librarian should disclose the status (available, borrowed, lost, damaged) of each copy as well.

A18: Librarians can manually modify the status of books in the library.

A19: The librarian can view a list of all pending borrow requests submitted by borrowers and manually approve the request.

A20: The librarian can view a list of pending return requests submitted by borrowers, manually manage these requests, and update the status of the related copies.

A21: All requests sent by the borrowers remain visible in the administration portal for historical retrieval and operational reference, even after they have been approved or completed.

A22: The system administrator can create new librarian accounts and view the information of librarians.

1.3 Non-Functional Requirements

1.3.1 Security

B1: User passwords are stored in a hashed form in the database.

B2: Login pages and protected endpoints are restricted to authenticated users only.

B3: Role-based authorization must be enforced to ensure only permitted roles can access specific system resources (e.g., book management pages are accessible to librarians only).

B4: Passwords must follow a strong complexity rule: they must meet minimum length requirements and include a mix of character types (such as uppercase and lowercase letters, digits, and special characters).

1.3.2 Usability

B5: The web interface should use a clear navigation menu with obvious links (Home, Browse Books, My Account, Librarian Dashboard, Admin Dashboard).

B6: After each operation, the system should provide informative feedback so that users clearly understand what has occurred and what actions are available next.

B7: Users should be able to easily reverse an action whenever appropriate.

1.3.3 Reliability and Data Integrity

B8: All critical operations (borrowing, returning, and account updates) should run within database transactions to keep data consistent.

1.3.4 Scalability

B9: The system follows a multi-tier architecture (Client, Web, Business, EIS tiers) to support horizontal or vertical scaling in the future.

B10: JPA abstracts the underlying database so the system can be migrated from MySQL to other relational databases if needed.

2. System Architecture Design

The system follows a classical four-tier Jakarta EE enterprise architecture, consisting of the Client Tier, Web Tier, Business Tier, and Enterprise Information System (EIS) Tier. This layered model enforces strict separation of concerns, enhances maintainability, and ensures that application logic, presentation logic, and persistent data operate within well-defined architectural boundaries. Each tier contributes a distinct set of responsibilities consistent with the component-based design principles taught in enterprise system development.

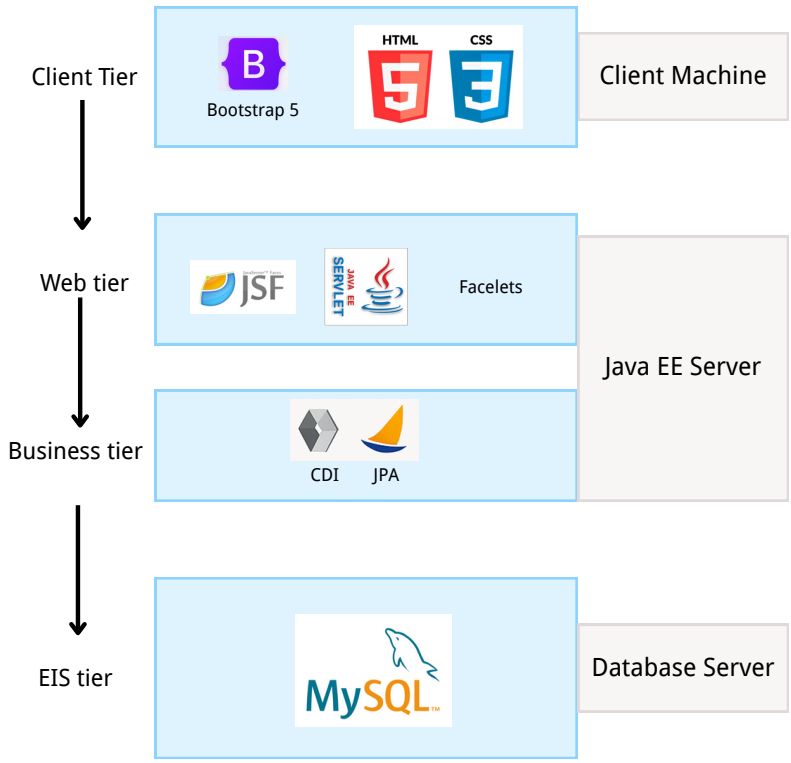


Figure 1: System Architecture

2.1 Client Tier

The client tier consists of the user-facing interface responsible for collecting input, presenting system responses, and enabling interaction across borrower, librarian, and administrator roles. This tier delivers responsive and accessible views styled with Bootstrap 5 and structured through server-rendered Facelets pages. It focuses purely on presentation and user experience, forwarding user actions to the server for processing while remaining free from business or persistence logic. Its primary responsibilities include displaying forms, tables, alerts, and navigation components that support the core workflows of the application.

2.2 Web Tier

The web tier manages all application interactions triggered by client requests. It handles request processing, input validation, session management, and navigation control. Using Jakarta Faces managed beans as controllers, this layer coordinates user actions and delegates all domain-specific operations to the business tier. By centralizing request handling and maintaining clean controller components, the web tier acts as an intermediary between the presentation layer and the underlying application logic, ensuring a structured and consistent user interaction model.

2.3 Business logic Tier

The business tier implements the functional core of the application. It encapsulates system rules such as login authentication, password validation, borrowing workflow management, return processing, overdue detection, and role-based operations. This layer employs CDI-managed services to organize domain logic into cohesive units that can be tested independently from the user interface. Transactional integrity is enforced to ensure that compound operations—such as borrow approvals or return finalization—execute with atomicity and consistency. Concentrating these operations within the business tier preserves clean separation of concerns and supports long-term extensibility.

2.4 Enterprise Information System (EIS) Tier

The EIS tier provides reliable and structured data management through Jakarta Persistence (JPA) and a MySQL relational database. Domain entities such as books, borrowers, librarians, copies, and borrowing records are mapped to relational tables through object-relational mapping, enabling standardized CRUD operations without exposing SQL directly to upper layers. This tier ensures data consistency, supports schema evolution, and enables the business tier to operate independently of database-specific concerns.

3. Component Design

The system follows a classic Java EE multi-tier architecture, composed of the Web Tier, Business Logic Tier, and Enterprise Information System Tier. The major components in each tier are described below, reflecting the actual packages and classes from the project structure.

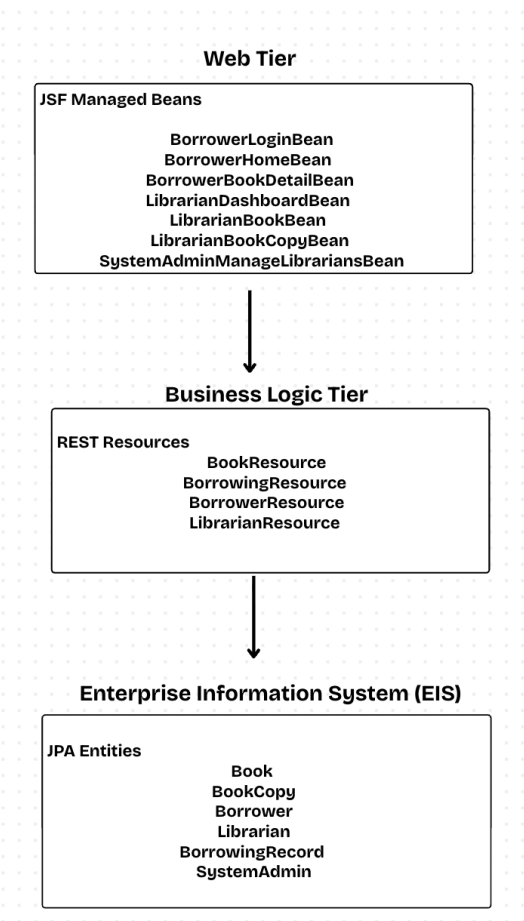


Figure 2: Java EE multi-tier architecture

3.1 Web Tier Components

Package: com.wangrui.librarymanagement.beans

The Web Tier is implemented using Jakarta Faces (JSF). It contains Managed Beans that act as controllers for the user interface. Each bean is responsible for handling page logic, processing form inputs, performing navigation, and interacting with the business logic resources.

Table 1: Borrower Beans

Managed Bean	Description
BorrowerHomeBean.java	Controls the borrower's homepage and dashboard content.
BorrowerLoginBean.java	Handles borrower authentication and login logic.
BorrowerRegisterBean.java	Manages borrower registration, validation, and account creation.
BorrowerProfileBean.java	Allows borrowers to view and update their personal account data.
BorrowerBookDetailBean.java	Displays book details and manages borrow requests submitted by borrowers.
BorrowerBorrowedBean.java	Lists currently borrowed books and manages return actions.

Table2: Librarian Beans

Managed Bean	Description
LibrarianLoginBean.java	Manages librarian login and session authentication.
LibrarianDashboardBean.java	Displays administrator statistics, pending borrow requests, and overall library status.
LibrarianBookBean.java	Handles CRUD operations for Book entities.
LibrarianBookCopyBean.java	Manages physical book copies, including status changes.
LibrarianBorrowerBean.java	Enables librarians to manage borrower accounts.
LibrarianBorrowingRecordBean.java	Creates and updates borrowing transactions.
LibrarianBorrowingHistoryBean.java	Displays borrowing logs, overdue records, and historical data.
LibrarianProfileBean.java	Manages the librarian's personal profile information.

Table 3: System Admin Beans

Managed Bean	Description
SystemAdminLoginBean.java	Handles system administrator authentication.
SystemAdminInitBean.java	Loads initial system configuration and administrative data.
SystemAdminManageLibrariansBean.java	Allows administrators to create, edit, and delete librarian accounts.

These components collectively form the controller layer of the system and directly link the user interface to backend operations.

3.2 Business Logic Tier Components

Package: com.wangrui.librarymanagement.resources

The Business Tier contains REST Resource classes that encapsulate the application's business logic. These components interact with JPA entities, perform validation, manage transactions, and expose programmatic interfaces for the JSF beans.

Table 4: REST Resource Components

Resource Class	Description
BookResource.java	Performs CRUD operations for Book entities.
BookCopyResource.java	Manages BookCopy creation, updates, and status control.
BookCopyStatusResource.java	Provides access to enumerated book copy statuses.
BookCategoryResource.java	Exposes operations for managing book categories.
BorrowerResource.java	Performs borrower account CRUD operations.
BorrowingResource.java	Handles borrowing and returning transactions, enforcing borrowing rules.
LibrarianResource.java	Manages librarian accounts and updates.
ApplicationConfig.java	Registers and configures REST endpoints for Jakarta EE 10.

3.3 Enterprise Information System (EIS) Tier Components

Package: com.wangrui.librarymanagement.entity

The EIS Tier contains the persistent domain model implemented using JPA. These Java classes map to relational database tables and represent the core data structures used throughout the system.

Table 5: Entity Classes

Entity Class	Description
Book.java	Represents a book title and contains metadata including title, author, ISBN, and category.
BookCategory.java	Represents classification categories for books.
BookCopy.java	Represents an individual physical copy of a book.
BookCopyStatus.java	Enumerates the possible states of a book copy (AVAILABLE, BORROWED, LOST, etc.).
Borrower.java	Represents a registered borrower account.
Librarian.java	Represents a librarian user.
SystemAdmin.java	Represents a system administrator user.
BorrowingRecord.java	Stores the borrowing transaction details including borrow date, due date, return date, and borrower information.

These classes define the system's persistent state and are automatically persisted and retrieved using JPA.

3.4 UML Class Diagram

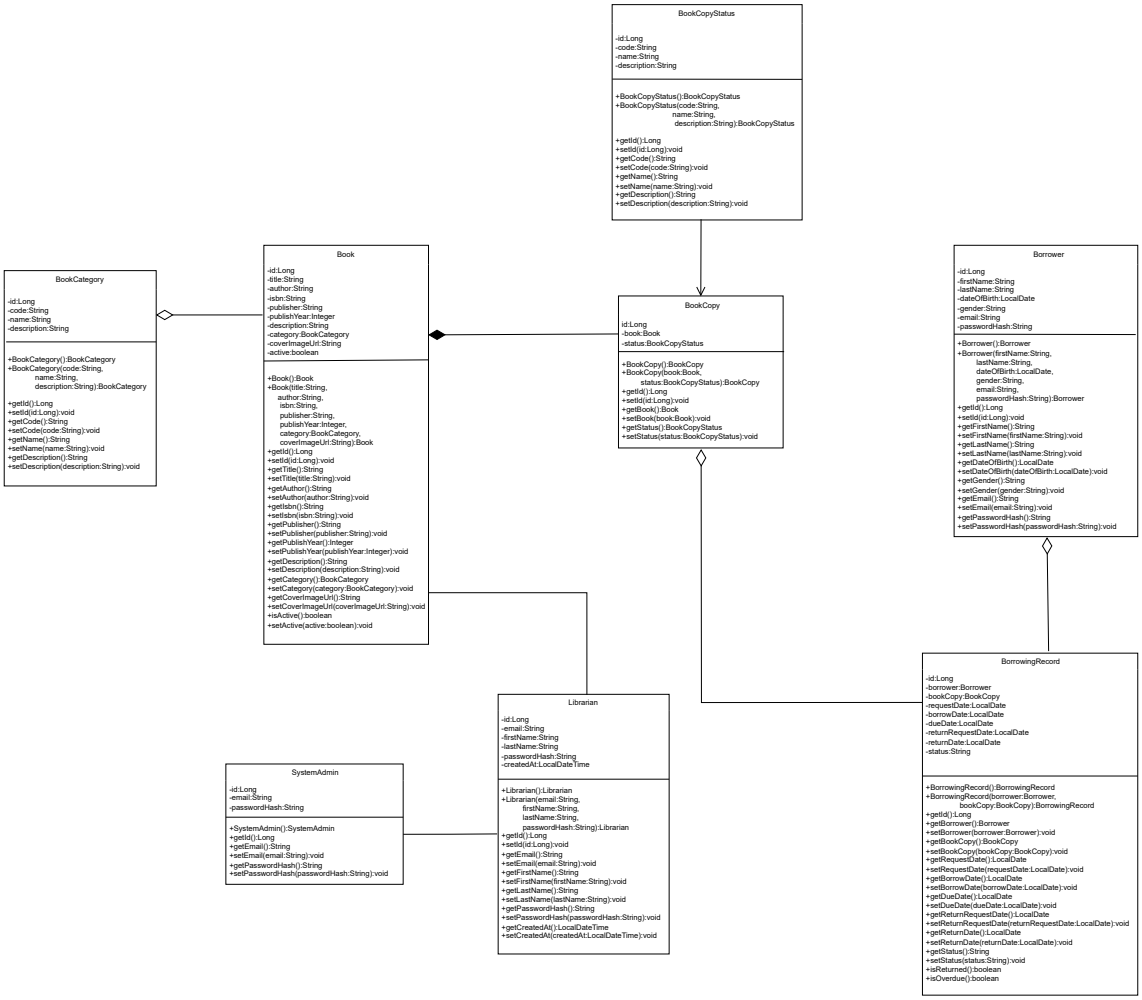


Figure 3: UML Class Diagram

4. Database Design

The database design of the Library Management System serves as the foundational layer supporting all core functionalities, including book catalog management, copy tracking, borrower authentication, and the complete borrowing workflow. A well-structured data model is essential to ensure integrity, consistency, and efficient access across the system's operations. To achieve this, the database architecture is constructed using a relational model that clearly distinguishes entities such as books, physical copies, borrowers, and borrowing records, while also defining the relationships that connect them. The Entity–Relationship (ER) diagram provides a high-level conceptual view of these entities and their interactions, illustrating cardinalities and constraints that govern data consistency. The schema design then translates this conceptual model into concrete database tables, specifying primary keys, foreign keys, attributes, and data types. Together, the ER diagram and schema design ensure that the system can accurately represent real-world library operations, enforce business rules, and support reliable, scalable data processing throughout the application lifecycle.

4.1 ER Diagram

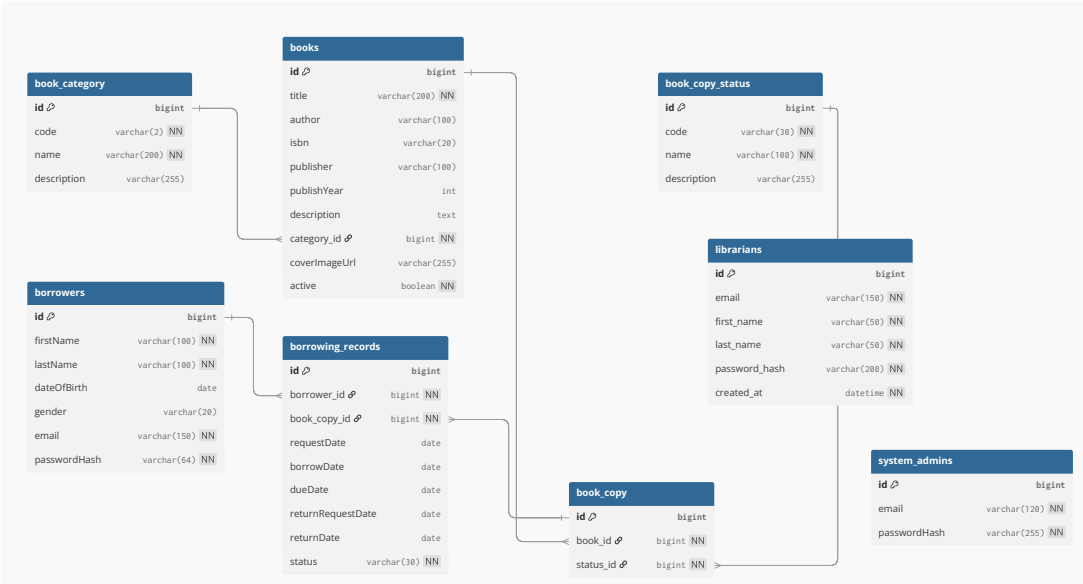


Figure 4: ER Diagram

4.2 Schema Design:

```
CREATE TABLE book_category (
    id            BIGINT       PRIMARY KEY AUTO_INCREMENT,
    code          VARCHAR(2)   NOT NULL UNIQUE,
    name          VARCHAR(200) NOT NULL,
    description   VARCHAR(255)
);

CREATE TABLE books (
    id             BIGINT        PRIMARY KEY AUTO_INCREMENT,
    title          VARCHAR(200)  NOT NULL,
    author         VARCHAR(100),
    isbn           VARCHAR(20),
    publisher      VARCHAR(100),
    publishYear    INT,
    description    TEXT,
    category_id    BIGINT        NOT NULL,
    coverImageUrl  VARCHAR(255),
    active         TINYINT(1)    NOT NULL,
    CONSTRAINT fk_books_category
        FOREIGN KEY (category_id)
        REFERENCES book_category(id)
);

CREATE TABLE book_copy_status (
    id            BIGINT        PRIMARY KEY AUTO_INCREMENT,
    code          VARCHAR(30)   NOT NULL UNIQUE,
    name          VARCHAR(100)  NOT NULL,
    description   VARCHAR(255)
);

CREATE TABLE book_copy (
    id         BIGINT   PRIMARY KEY AUTO_INCREMENT,
    book_id    BIGINT   NOT NULL,
    status_id  BIGINT   NOT NULL,
    CONSTRAINT fk_bookcopy_book
        FOREIGN KEY (book_id)
        REFERENCES books(id),
    CONSTRAINT fk_bookcopy_status
        FOREIGN KEY (status_id)
        REFERENCES book_copy_status(id)
);

CREATE TABLE borrowers (
    id            BIGINT        PRIMARY KEY AUTO_INCREMENT,
    firstName     VARCHAR(100)  NOT NULL,
    lastName      VARCHAR(100)  NOT NULL,
    dateOfBirth   DATE,
    gender        VARCHAR(20),
    email         VARCHAR(150)  NOT NULL UNIQUE,
    passwordHash  VARCHAR(64)   NOT NULL
);

CREATE TABLE borrowing_records (
    id                BIGINT       PRIMARY KEY
AUTO_INCREMENT,
    borrower_id       BIGINT       NOT NULL,
    book_copy_id      BIGINT       NOT NULL,
    requestDate       DATE,
    borrowDate        DATE,
    dueDate           DATE,
    returnRequestDate DATE,
    returnDate        DATE,
    status            VARCHAR(30)  NOT NULL,
    CONSTRAINT fk_borrowing_borrower
        FOREIGN KEY (borrower_id)
        REFERENCES borrowers(id),
    CONSTRAINT fk_borrowing_bookcopy
        FOREIGN KEY (book_copy_id)
        REFERENCES book_copy(id)
);

CREATE TABLE librarians (
    id             BIGINT        PRIMARY KEY AUTO_INCREMENT,
    email          VARCHAR(150)  NOT NULL UNIQUE,
    first_name     VARCHAR(50)   NOT NULL,
    last_name      VARCHAR(50)   NOT NULL,
    password_hash  VARCHAR(200)  NOT NULL,
    created_at     DATETIME      NOT NULL
);

CREATE TABLE system_admins (
    id            BIGINT        PRIMARY KEY AUTO_INCREMENT,
    email         VARCHAR(120)  NOT NULL UNIQUE,
    passwordHash  VARCHAR(255)  NOT NULL
);
```

Figure 5: schema

5. Interface Design

5.1 Site map

The site map displays the overall structure of book management system.

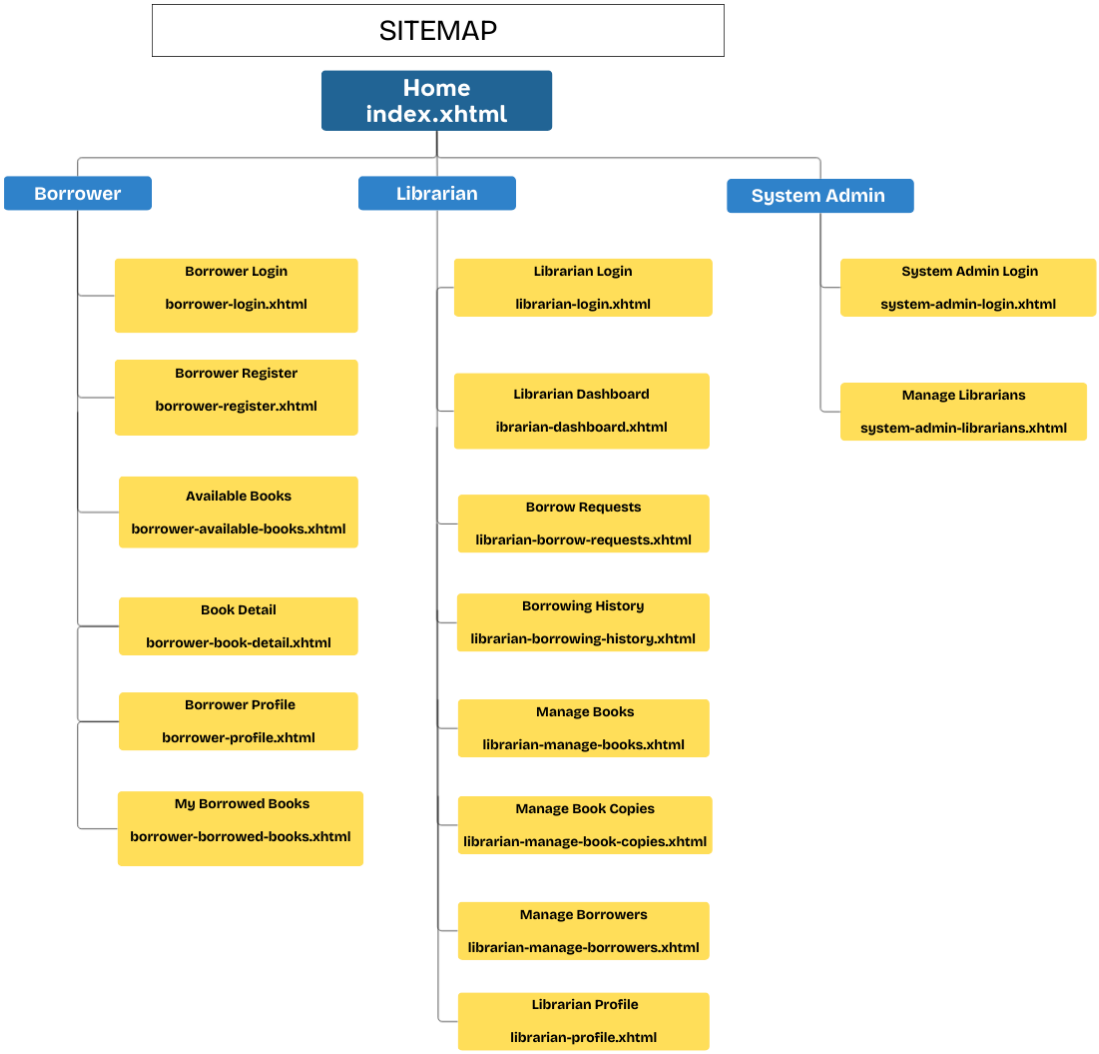


Figure 6: sitemap

5.2 HCI principle

5.2.1 Strive for consistency

The interface design of the library management system consistently applies a unified visual and interaction style across different pages. This includes maintaining the same layout structure, typography, navigation patterns, and overall appearance throughout the system. With these consistent design

conventions, users can easily predict where information and controls will be located, which helps them interact with the system more efficiently.

By reducing unexpected variations between pages, the interface minimizes the need for users to learn new layouts or remember temporary details while navigating repeatedly. As a result, the cognitive load—especially short-term memory burden—is significantly reduced, allowing users to focus more on completing their tasks rather than figuring out how to use the interface. This consistency enhances usability and supports a smoother, more intuitive user experience.

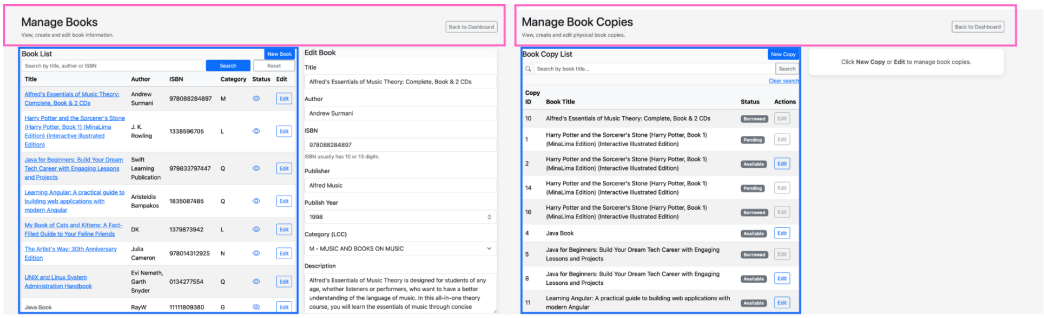


Figure 7: Interface of book management and book copy management (consistent style)

5.2.2 Offer informative feedback

The system provides clear and informative feedback to guide users during critical interactions such as registration and login. When a new borrower completes the registration process successfully, the interface displays a prominent success message, informing the user that the account has been created, and they may now proceed to log in. This immediate positive feedback reassures the user that their action was completed correctly and helps them understand the next step.

Similarly, if a login attempt fails—for example, due to an incorrect email or password—the system presents a clear error message above the form. This notification allows the user to quickly identify what went wrong and correct the issue without confusion or repeated failed attempts. By providing timely and meaningful responses to user actions, the system reduces uncertainty, improves task efficiency, and supports a more guided and user-friendly interaction flow.

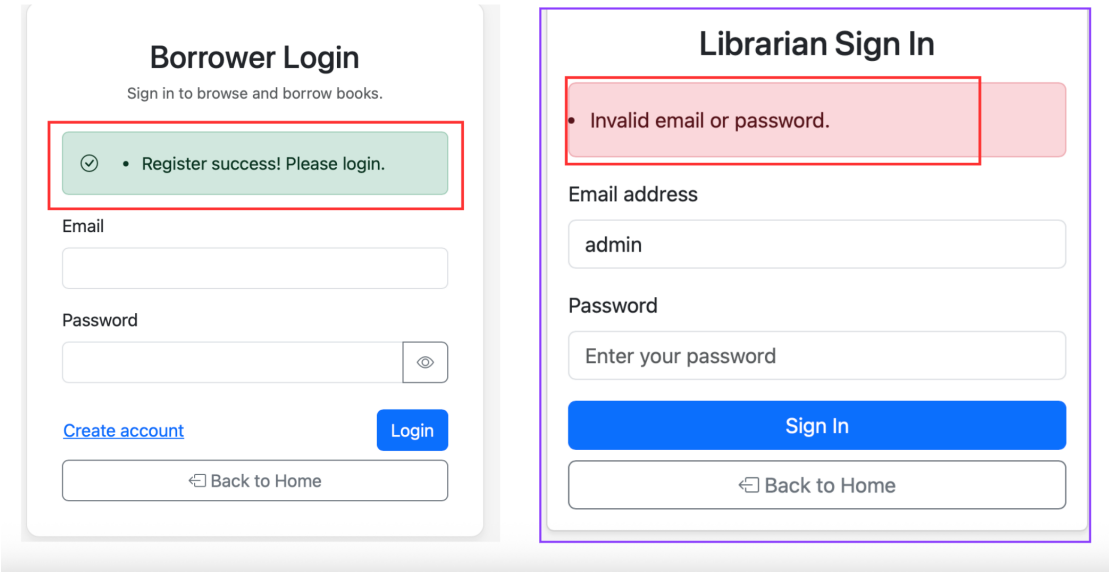


Figure 8: Interactive informative feedback

5.2.3 Prevent errors

The interface demonstrates effective error-prevention by clearly informing users of missing or incorrect inputs before final submission. During borrower registration, required fields such as last name, email, and password are highlighted with a visible notification box, allowing users to recognize what needs completion at an early stage. Additionally, inline validation messages appear directly beneath each form field, such as “Password is required” and “Please confirm your password.”, reducing the chance of overlooked information. This real-time feedback guides users step by step, preventing mistakes rather than only reporting them afterward. By presenting errors prominently and contextually, the system minimizes user frustration and improves the overall correctness and efficiency of form completion.

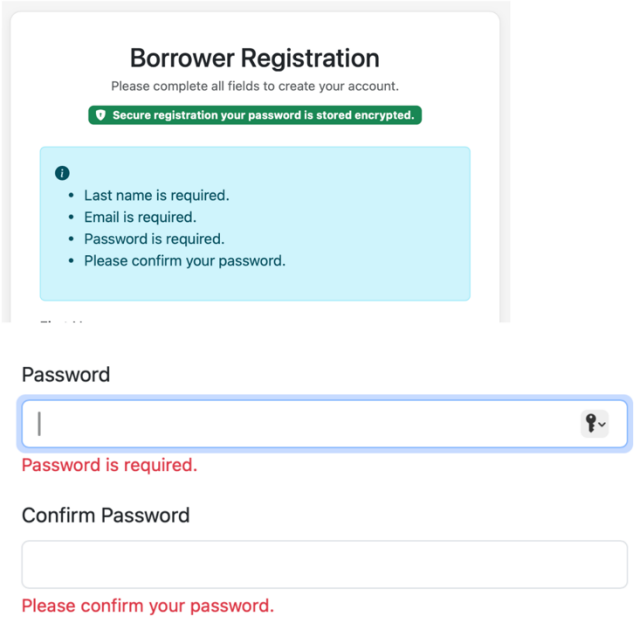


Figure 9: Informative Error reasons

5.2.4 Permit easy reversal of actions

The interface effectively supports easy reversal of actions by offering users clear and consistent ways to navigate backward or undo a navigation step without penalty. Across multiple pages—including the Librarian Sign In, Borrower Login, and Borrow/Return Requests screens—prominently placed “Back to Home” or “Back to Dashboard” controls allow users to return to a previous safe and familiar state at any time.

By making reversal actions immediately accessible and visually distinct, the system reduces the cognitive load on users and lowers the fear of making a wrong choice. Users are not forced into irreversible paths or multi-step navigation loops, which decreases stress and encourages exploration of the system. This design approach aligns with usability best practices by allowing users to comfortably correct navigation decisions and recover from mistakes with minimal effort, ultimately enhancing confidence, efficiency, and ease of use.

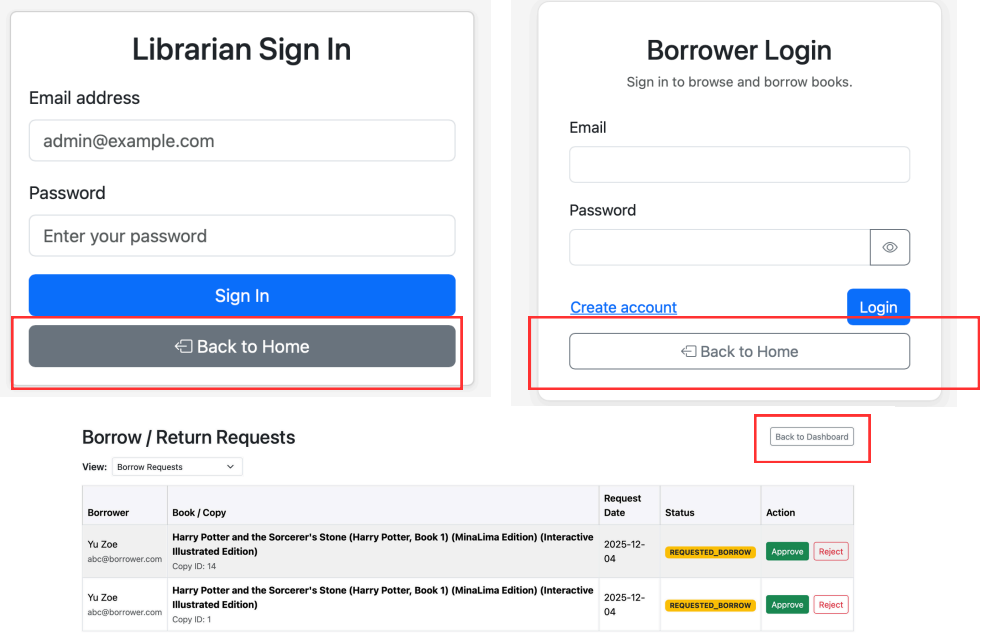


Figure 10: Reversal button

5.3 Sequence Diagrams

5.3.1 Sequence Diagram 1 — Borrower Submits Borrow Request

This sequence diagram illustrates the workflow of how a borrower initiates a borrow request for a specific book copy. When the borrower clicks the “Request Borrow” button on the book detail page, the system updates the selected book copy status to PENDING and creates a new borrowing record marked as REQUESTED_BORROW. This process involves several system components, including the JSF page, managed bean, business logic service, and the database layer. The purpose of this diagram is to demonstrate how the system validates availability, creates the pending transaction, and ensures that borrower requests are properly stored and forwarded to the librarian for further approval.

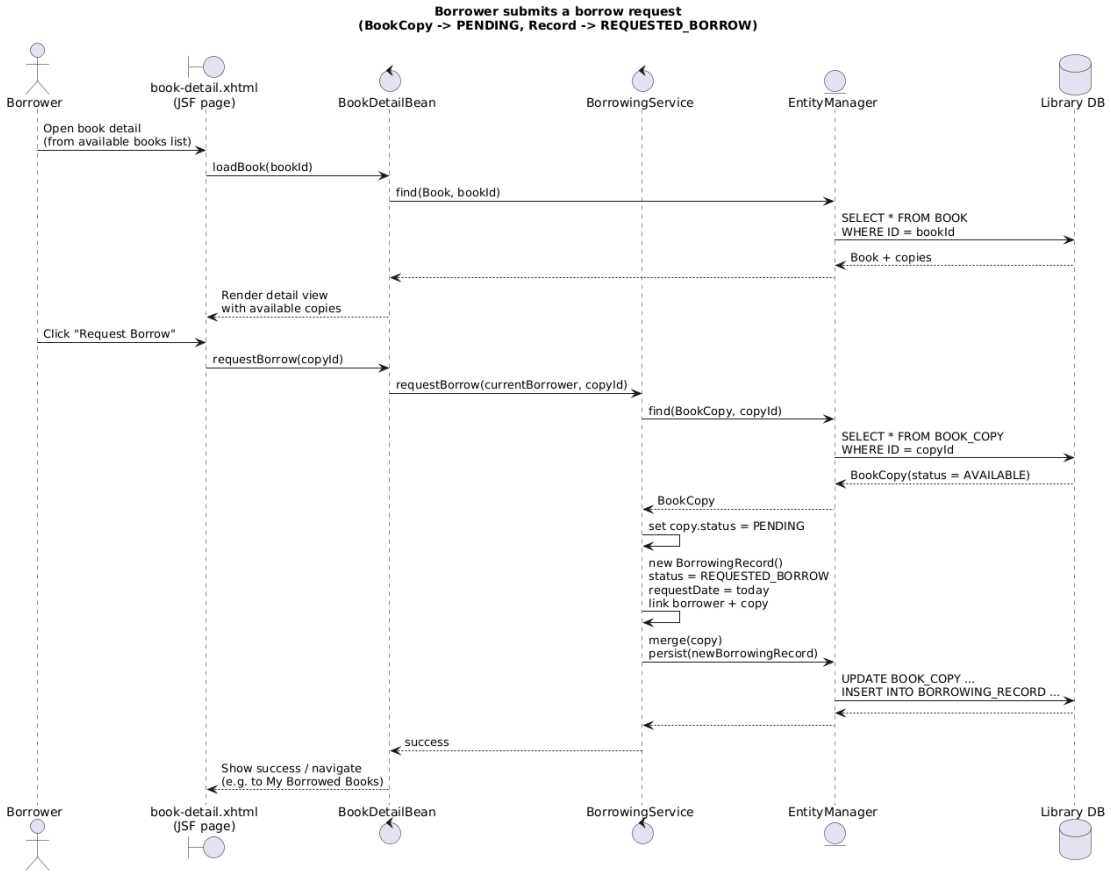


Figure 11: Borrower Submits Borrow Request diagram

5.5.2 Sequence Diagram 2 Introduction — Librarian Approves or Rejects Borrow Request

This sequence diagram explains how the librarian processes a borrower's pending borrow request. When the librarian opens the request management interface, the system retrieves all REQUESTED_BORROW borrowing records. The librarian may choose to either approve or reject a specific request. Approving the request updates the book copy status to BORROWED and marks the borrowing record as BORROWED, also initializing the 14-day loan period. Rejecting the request restores the book copy to AVAILABLE and sets the borrowing record status to INCOMPLETE. This diagram highlights the decision-making flow and demonstrates how administrative actions trigger state transitions in both copies and borrowing records.

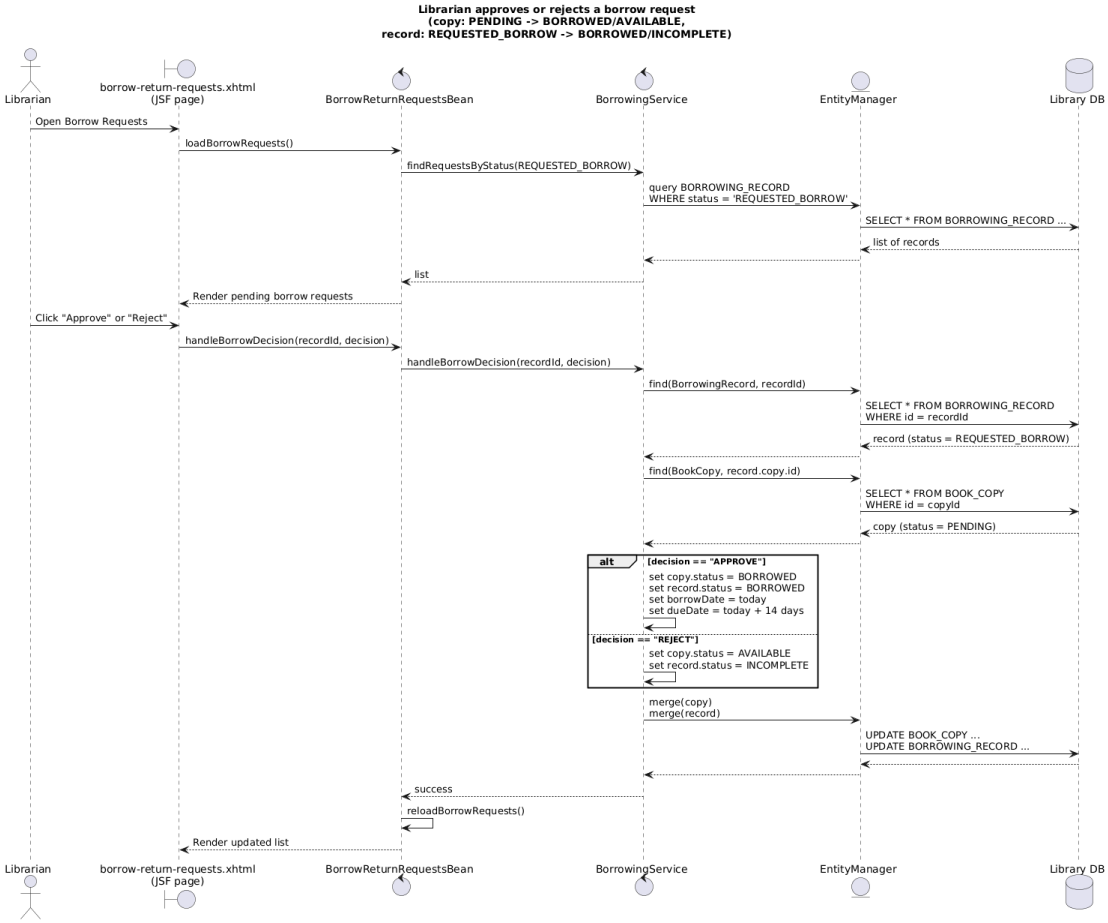


Figure 12: Librarian Approves or Rejects Borrow Request diagram

5.5.3 Sequence Diagram 3 Introduction — Borrower Return Request, Librarian Confirms Return, and Auto-Overdue Handling

This sequence diagram describes the complete workflow of returning a borrowed book. First, the borrower initiates a return request, which updates the borrowing record status to RETURN_REQUESTED while keeping the book copy in BORROWED state. Next, the librarian reviews pending return requests and confirms the physical condition of the returned book. Depending on whether the book is intact, damaged, or lost, the system updates the book copy status accordingly and marks the borrowing record as either COMPLETED or INCOMPLETE. The diagram also includes the automated overdue process, where the system periodically checks for records that exceed the 14-day loan limit and automatically marks both the book copy and borrowing record as LOST and INCOMPLETE. This provides a full representation of return handling and overdue management within the system.

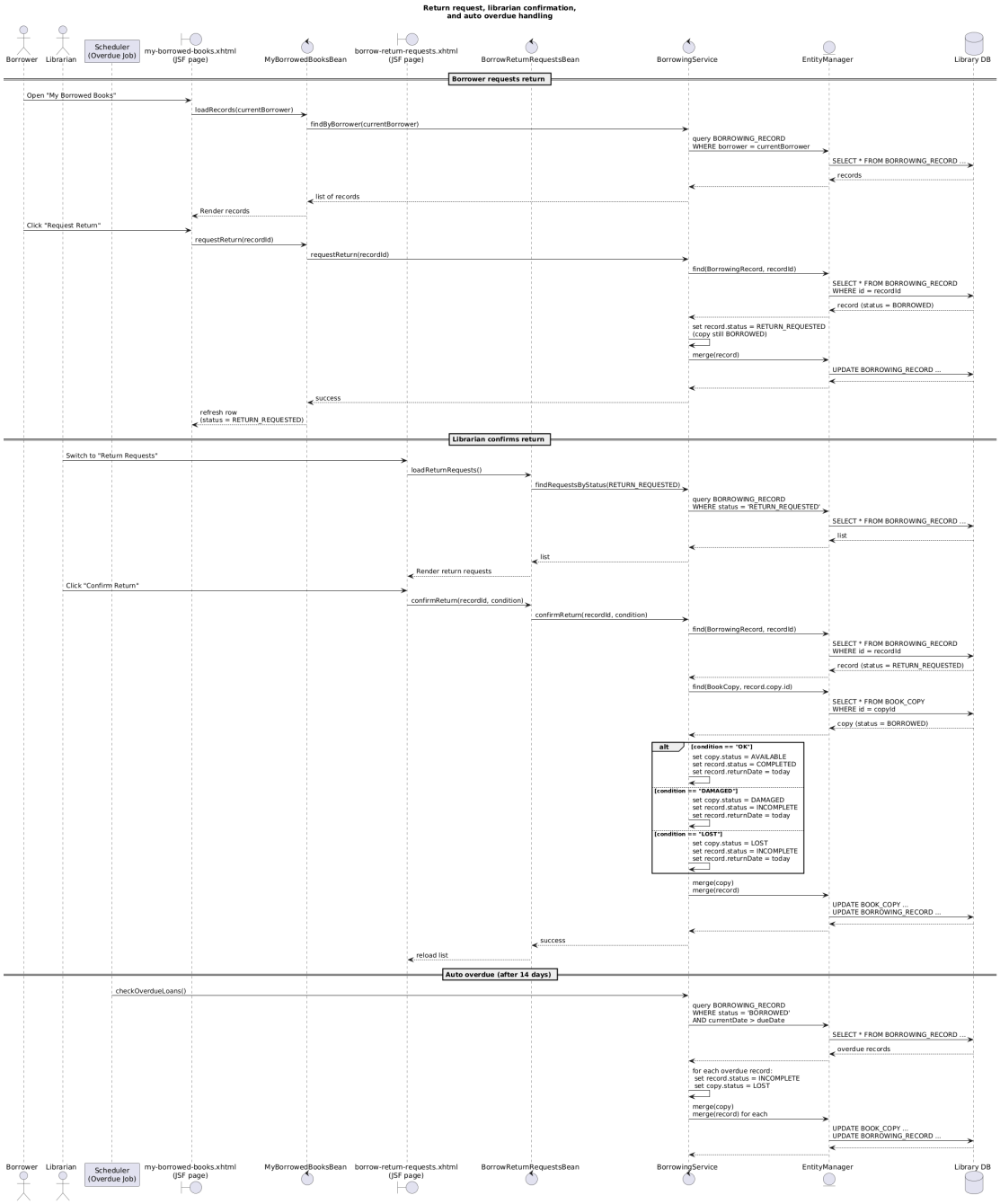


Figure 13: Borrower Return Request, Librarian Confirms Return, and Auto-Overdue Handling diagram

5.4 Interface disclosure

This section presents the different interfaces designed for distinct user groups. For the general public, the system provides access to basic library functions such as book search and book information display. However, users must register as borrowers before they are allowed to borrow books, ensuring that all transactions are properly recorded, traceable, and managed securely.

For the librarian portal, each staff member is assigned a designated work email and a default passcode to access the system and perform their responsibilities. Librarians also have the ability to update their passwords at any time to maintain account security and support long-term usability.

5.4.1. General Public

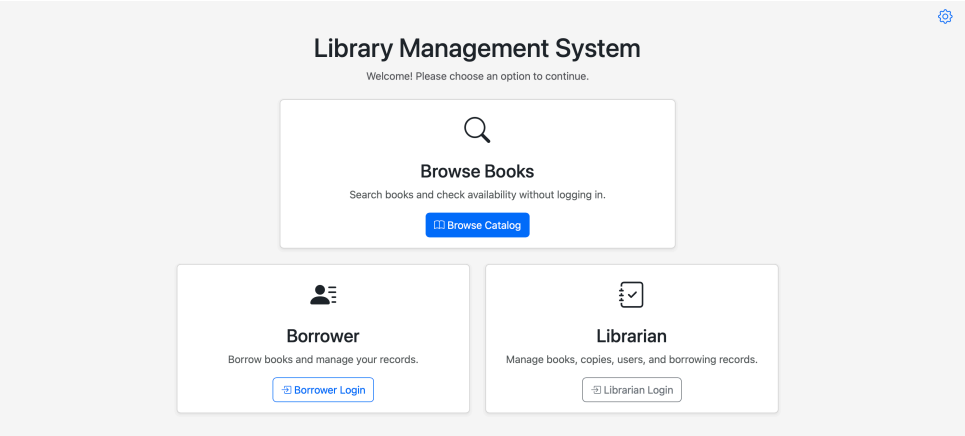


Figure 14: Home Page

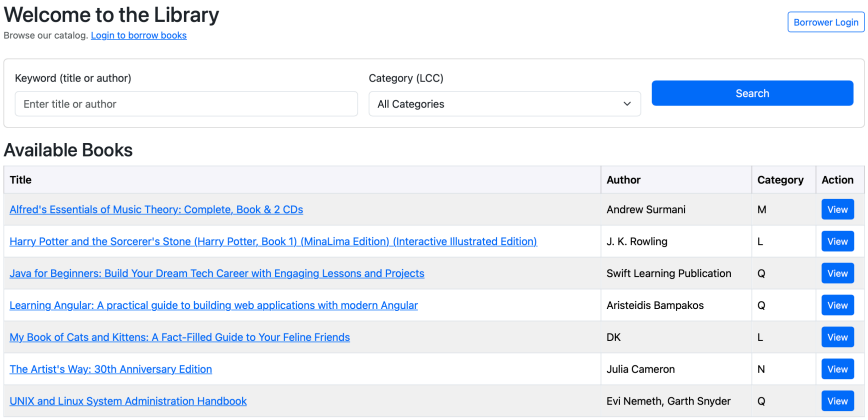


Figure 15: Book list Page

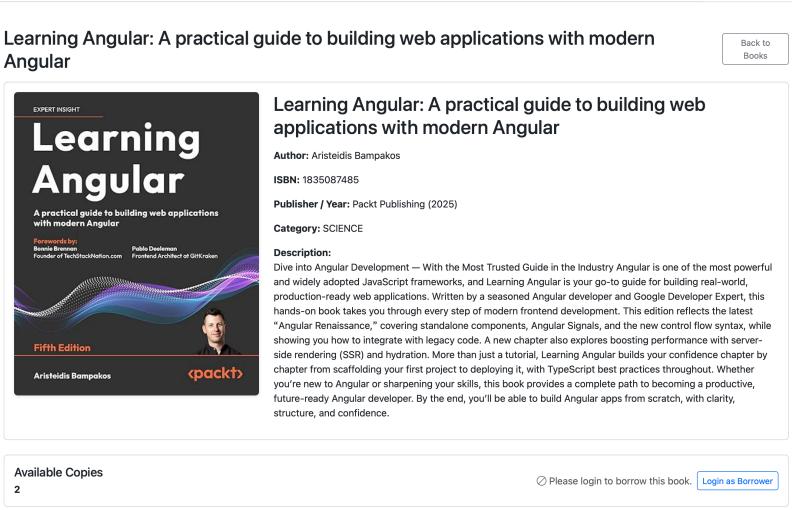


Figure 16: Book detail Page

The following are the interfaces that are visible to **users(borrowers).**

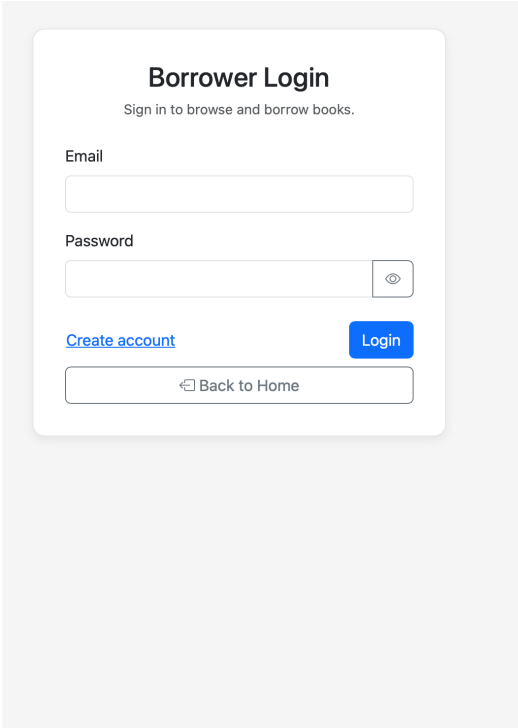


Figure 17: Borrower login Page

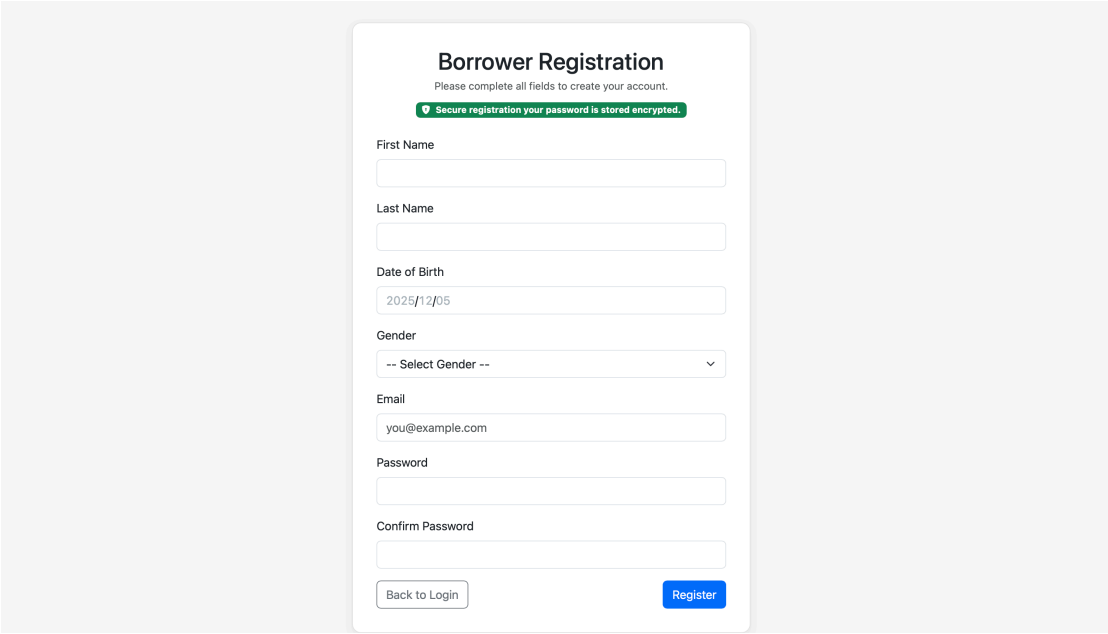


Figure 18: Borrower Register Page

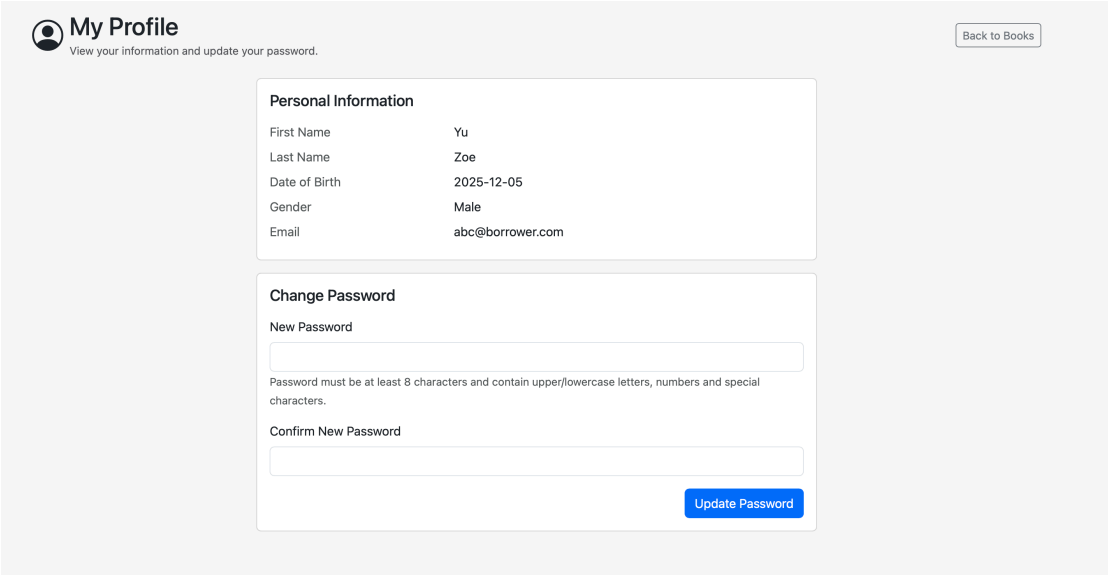


Figure 19: Borrower Profile Page

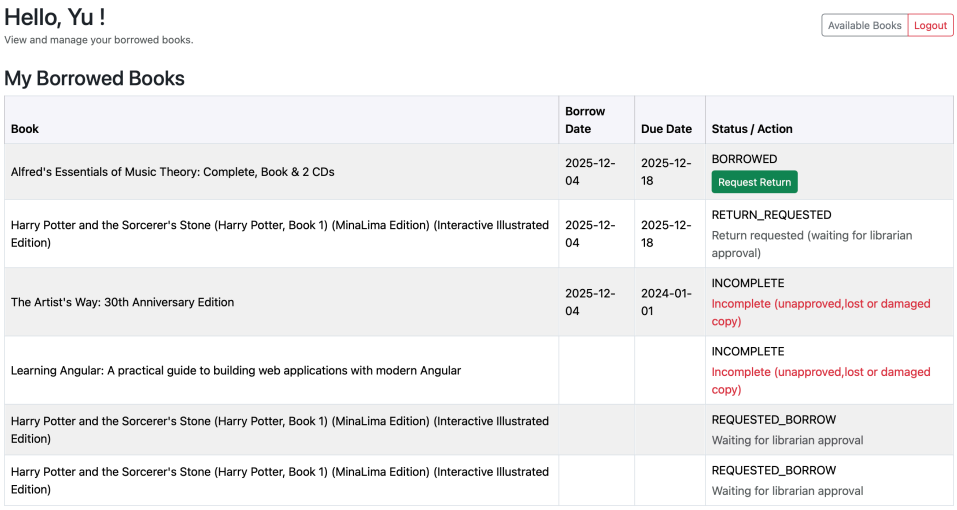


Figure 20: Borrowing record page (Borrower)



Figure 21: Book detail page(Borrower can request a book)

5.4.2 Administrators

The following are the interfaces that are visible to administrators (Librarians and system administrators).

Librarian

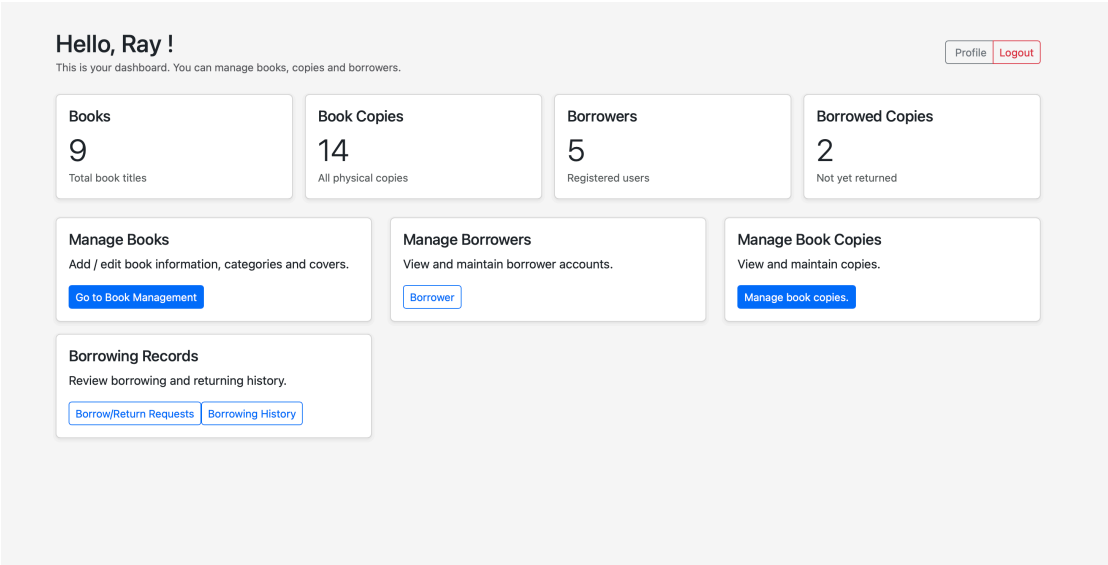


Figure 22: Dashboard page of Librarian

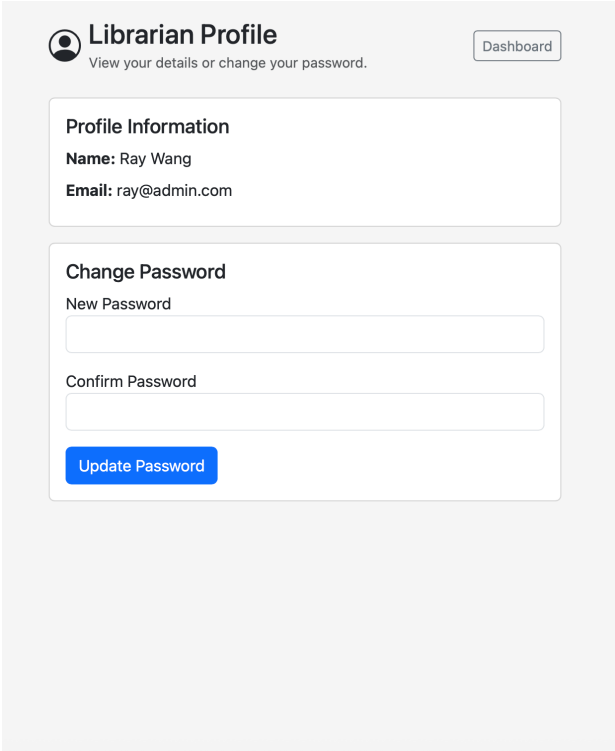


Figure 23: Profile page of Librarian

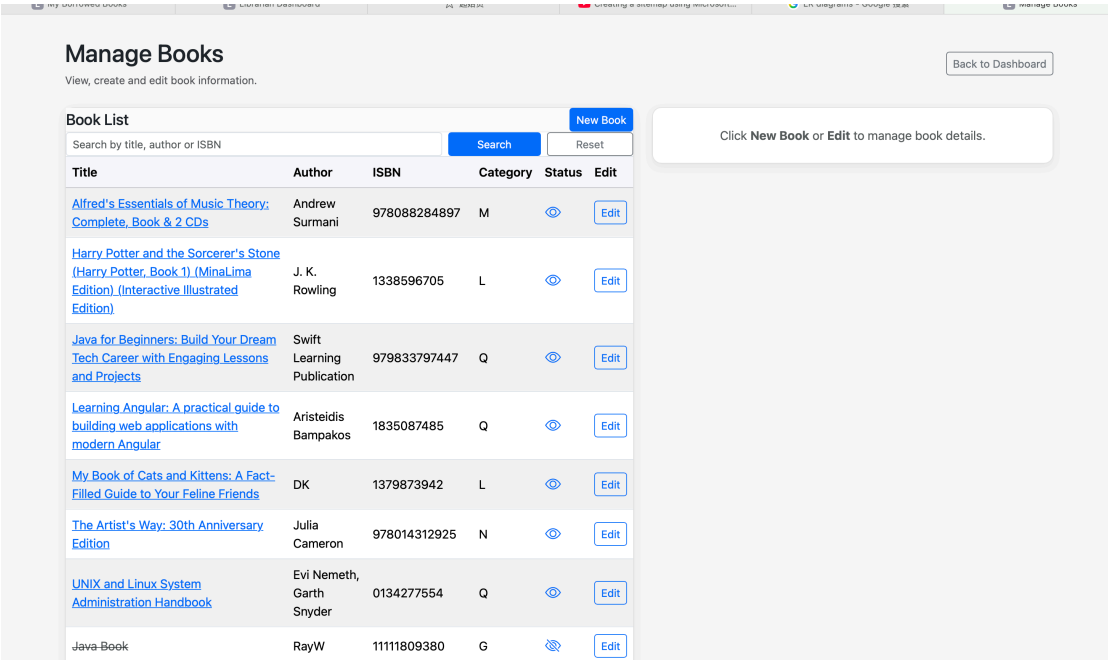


Figure 24: Book management page

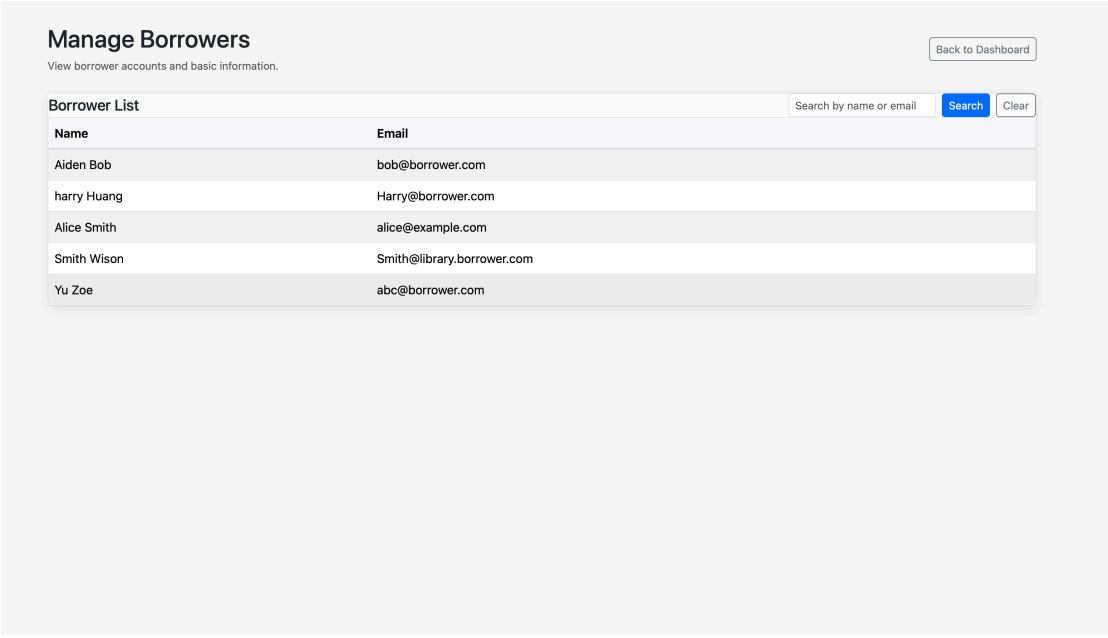


Figure 25: Borrower management page

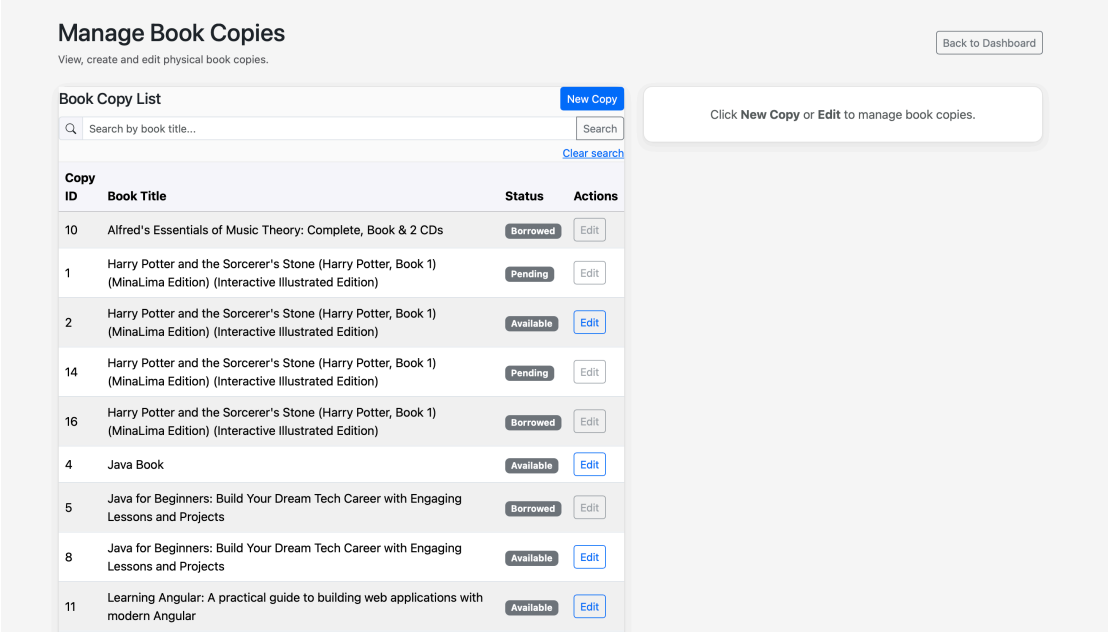


Figure 26: Book copies management page

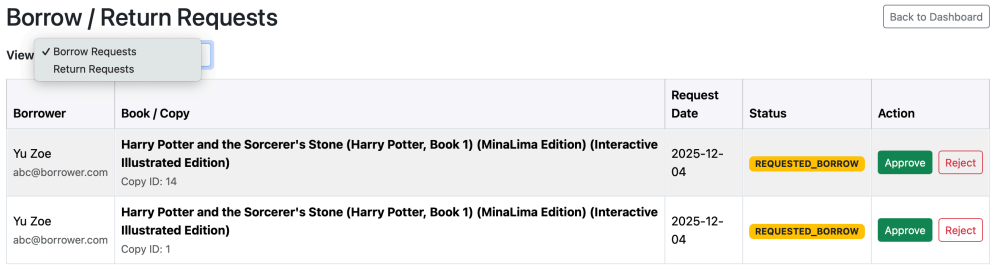


Figure 27: Borrow requests page

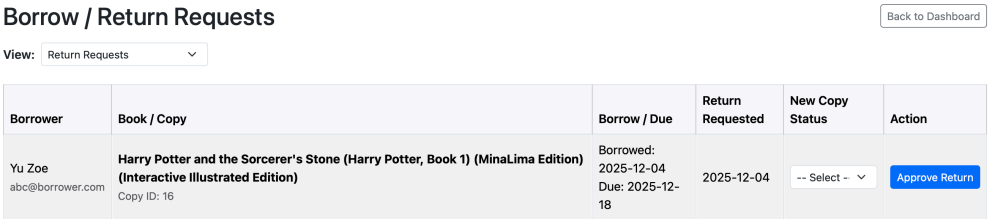


Figure 28: Return requests page

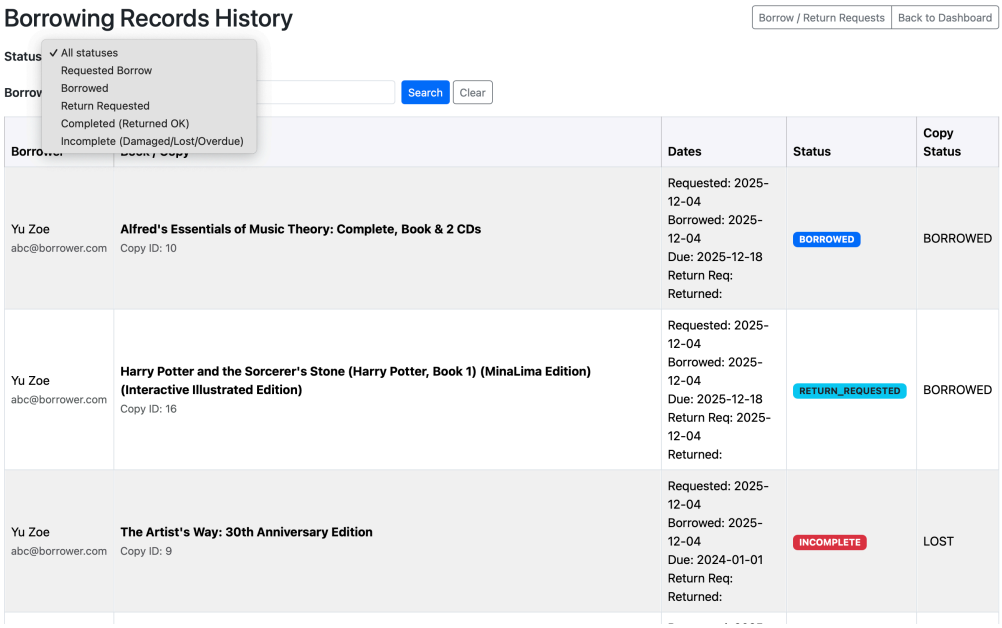


Figure 29: Borrowing Records History (All status)

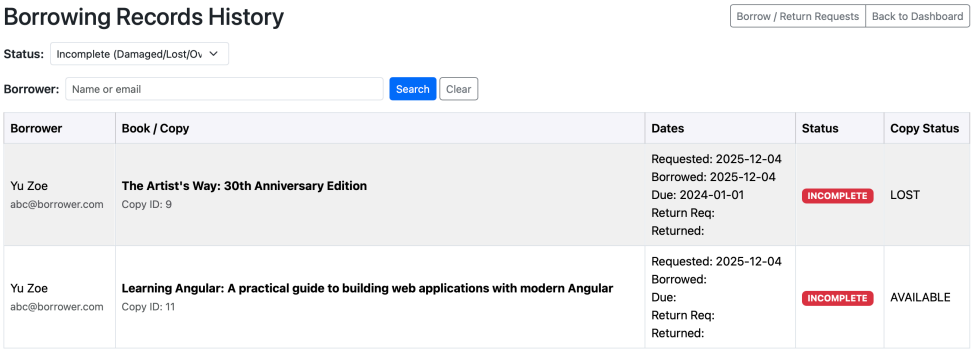


Figure 30: Borrowing Records History (Incomplete)

System admin

For the system administrator, we have currently only granted them the permission to access the information of the librarian. However, this can be scaled in the future.

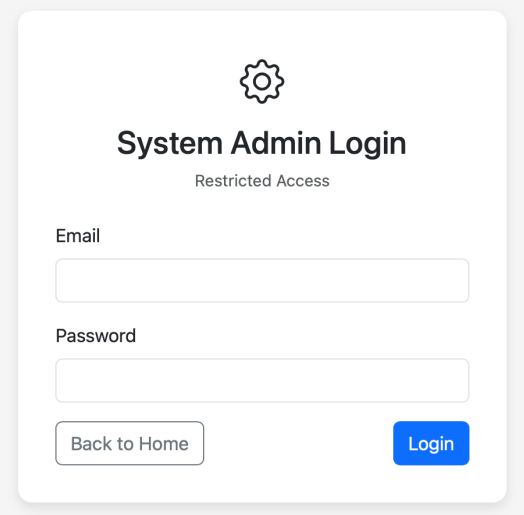


Figure 31: System Admin login

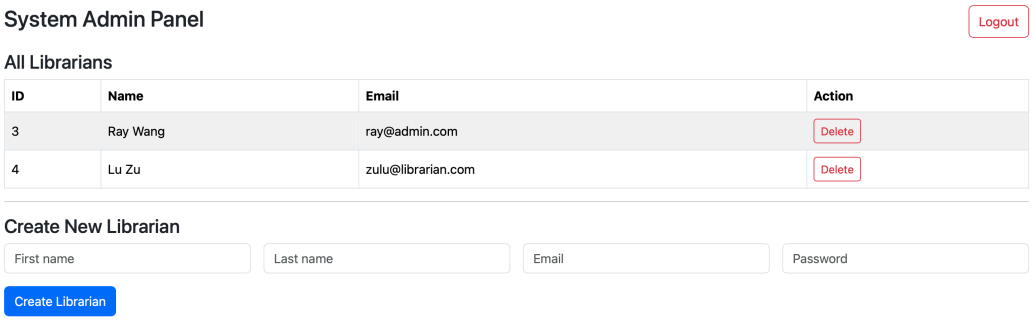


Figure 32: System Admin manages librarians

5.4.3 Advanced feature:

This project incorporates several advanced features that enhance security, reliability, and robustness beyond the core functional requirements. These improvements focus on password protection, access control, and ensuring that sensitive operations are strictly regulated at both the interface and system levels.

To ensure the security of user credentials, the system implements bcrypt hashing for all stored passwords, including borrower and librarian accounts. Instead of saving plaintext credentials, each password is transformed into a salted hash—visible in the database as long encrypted strings.

```
1 •  USE librarydb;
2
3 •  select * from borrowers;
```

100% 25:3

Result Grid Filter Rows: Search Edit: Export/Import:

ID	DATEOFBIRTH	EMAIL	FIRSTNAME	GENDER	LASTNAME	PASSWORDHASH
1	1995-05-10	alice@example.com	Alice	NULL	Smith	$2a$10$Gl80GNT5YGIO5FjQneEYoOBcMhU4…
2	2025-12-05	Smith@library.borrower.com	Smith	Male	Wison	$2a$10$6mM.M95HPV0r5SatMqAvl.DwlCWYp…
3	2025-12-05	abc@borrower.com	Yu	Male	Zoe	$2a$10$ok8R7dOmlmDeFhKDjbHCIuZSGIL0y…
4	2000-02-02	bob@borrower.com	Aiden	Other	Bob	$2a$10$7tbFfwR7fdqtPw1fiG7Ka.4FFdbK3Ive5…
5	1999-02-20	Harry@borrower.com	harry	Male	Huang	$2a$10$cbK6MVQXd6K2.QivUdEL5uSoRFQT…
6	2025-12-01	ai@borrower.com	Ai	Female	R	$2a$10$uxQhGin31UADzfhSQBsjreek/4g2I4gj…

Figure 33: Entity in database

To further enhance account security, the system enforces a strict password complexity policy during borrower registration and modification. Users must create passwords that contain at least eight characters and include a combination of uppercase letters, lowercase letters, digits, and special characters. If the entered password fails to meet these requirements, the registration interface provides immediate inline feedback indicating the specific rule that has not been satisfied. This real-time validation prevents the creation of weak or easily guessable passwords and guides users toward choosing more secure credentials. Through this mechanism, the system ensures that all borrower accounts are protected with strong authentication practices that reduce the likelihood of unauthorized access.

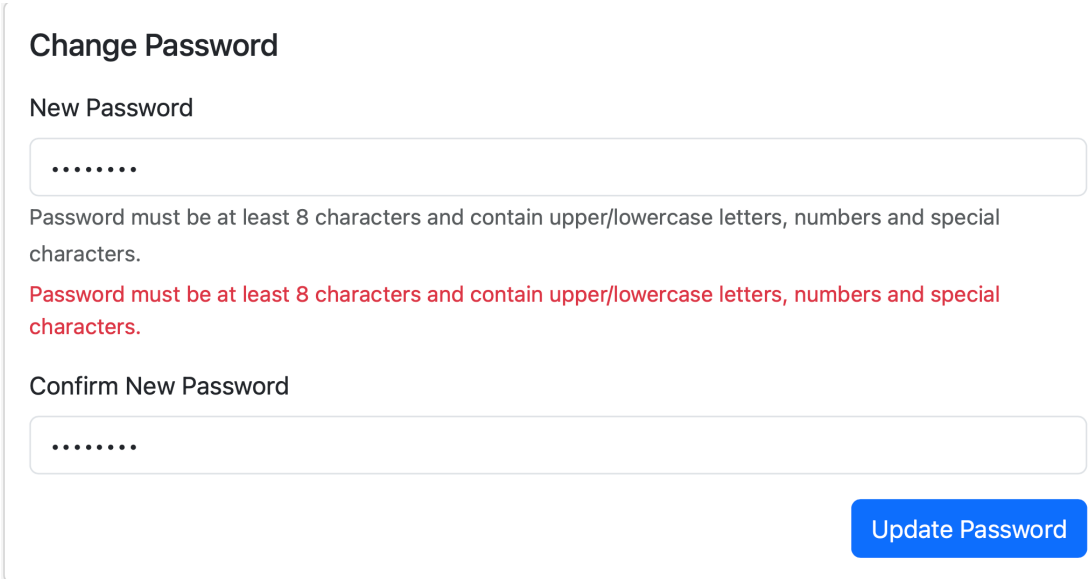


Figure 34: Passcode security

The system incorporates a comprehensive access control mechanism that assigns specific permissions to different types of users, including public visitors, registered borrowers, and librarians. Public users may browse book information but cannot request loans or view borrower-only pages. Registered borrowers gain access to additional capabilities such as borrowing books, viewing their borrowing history, and managing their profiles, yet they remain restricted from library management functions. Librarians, on the other hand, are granted full administrative access, enabling them to manage book catalogs, edit copy information, and process borrowing and return requests. All sensitive pages perform server-side validation to confirm that users have authenticated sessions with the correct role assignment. This layered authorization framework ensures that confidential data and administrative functionality are accessible only to the appropriate personnel, thereby maintaining both data integrity and operational security.

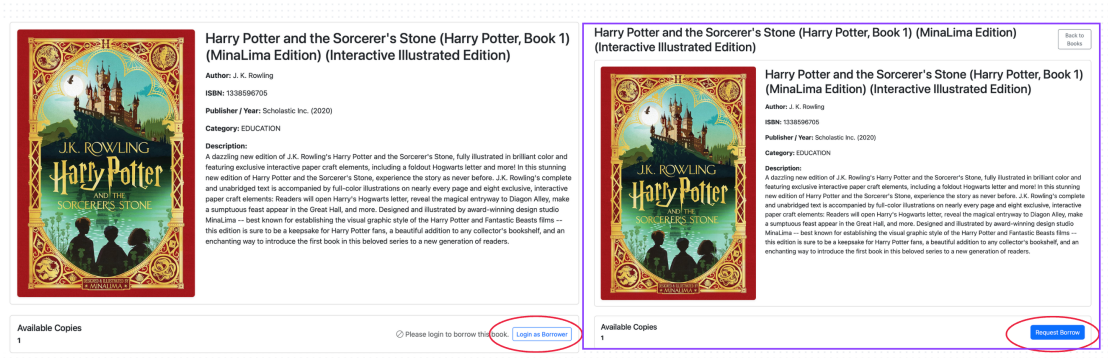


Figure 35: Display changes of different identities

To prevent unauthorized users from viewing protected pages, the system includes automatic redirection mechanisms that guide users back to the correct authentication interface whenever they attempt to access restricted content. For instance, a borrower attempting to reach a librarian-only management page is redirected to the librarian login page, while an unauthenticated visitor trying to open any protected area is likewise brought to the appropriate login screen. This feature ensures that users cannot bypass authentication simply by entering a URL manually. By providing clear redirection paths, the system maintains a secure navigation flow, prevents exposure of restricted data, and minimizes user confusion by indicating the correct method for accessing protected resources. This seamless integration of security and usability contributes to a safer and more intuitive user experience.

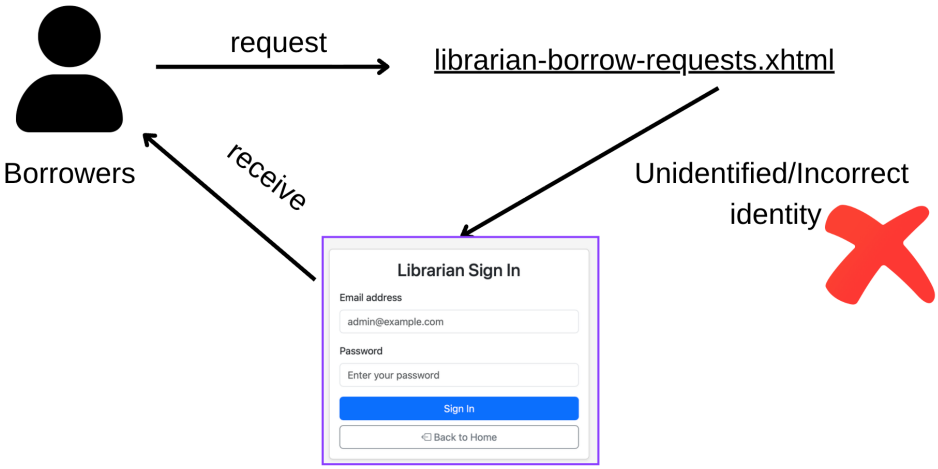


Figure 36: Tests for status transformation

6. Testing

A rigorous testing strategy was adopted to validate the functional correctness, reliability, and robustness of the Library Management System. In this project, multiple layers of tests were designed and executed, encompassing utility-level validation, authentication logic, workflow business processes, and time-dependent overdue mechanisms. The testing approach combines JUnit 5, Mockito, and service-level simulation to ensure comprehensive verification without requiring a full server deployment.

6.1 Unit Testing

Unit tests were developed using JUnit 5 and Mockito, allowing isolated execution of business logic without requiring a running server or database. The testing work primarily focused on core utility classes and authentication processes.

Tests for the PasswordUtil class verify that password hashing uses salted hashing, that correct passwords can be validated, and that incorrect or malformed inputs fail safely. Strong password rules—requiring minimum length, uppercase letters, lowercase letters, digits, and special characters—are also validated to ensure compliance with security standards.

For the BorrowerLoginBean, Mockito was used to mock both the EntityManager and the JSF FacesContext to simulate realistic authentication scenarios. These tests cover successful login, incorrect passwords, unknown email addresses, blank or null inputs, multiple users with the same email, missing password hashes, and extremely long password strings. This guarantees that authentication behaves correctly across a wide range of valid and invalid conditions.

6.2 Integration Testing

To approximate integration testing in a lightweight environment, domain logic was encapsulated into a dedicated service class (BorrowingWorkflowService), enabling the borrowing workflow to be tested independently of container infrastructure. The service-level tests evaluate the correctness of the full borrowing lifecycle.

The BorrowingWorkflowServiceTest simulates the complete lifecycle of borrowing operations:

- submitting a borrow request
- approving or rejecting a request
- generating due dates
- initiating a return request
- finalizing the return with different outcomes (AVAILABLE, DAMAGED, LOST)

These tests validate how different entities (BookCopy, BorrowingRecord, Borrower) interact together, ensuring end-to-end correctness of borrowing logic. The tests also confirm proper error handling—for example, rejecting requests when no available copies exist, or throwing exceptions when required inputs are missing.

In addition, the OverdueUtilTest verifies the system's overdue handling mechanism. Tests confirm correct detection of overdue items, and automatic conversion of unreturned, past-due records into LOST status while appropriately marking their associated borrowing records as INCOMPLETE. This time-dependent logic is tested thoroughly to ensure reliability.

6.3 Edge Case Handling

A significant effort was dedicated to ensuring robustness through explicit edge-case testing.

The system is tested against:

- null or empty strings
- missing password hashes
- duplicated database results
- absence of available book copies
- extremely long passwords
- past-due dates with missing return dates
- attempts to borrow without proper user authentication

These scenarios reflect real-world issues encountered in production systems and demonstrate defensive programming practices.

6.4 Bug Resolution Process

Throughout development, several logical issues were identified and corrected based on test outcomes.

Examples include:

- preventing login attempts from bypassing session assignment
- ensuring null password hashes do not cause exceptions
- avoiding incorrect state transitions in borrowing workflows
- enforcing valid password rules before account creation
- handling overdue items consistently and automatically

The testing process not only validated existing functionality but also contributed to improved code quality, reliability, and maintainability.

6.5 Conclusion

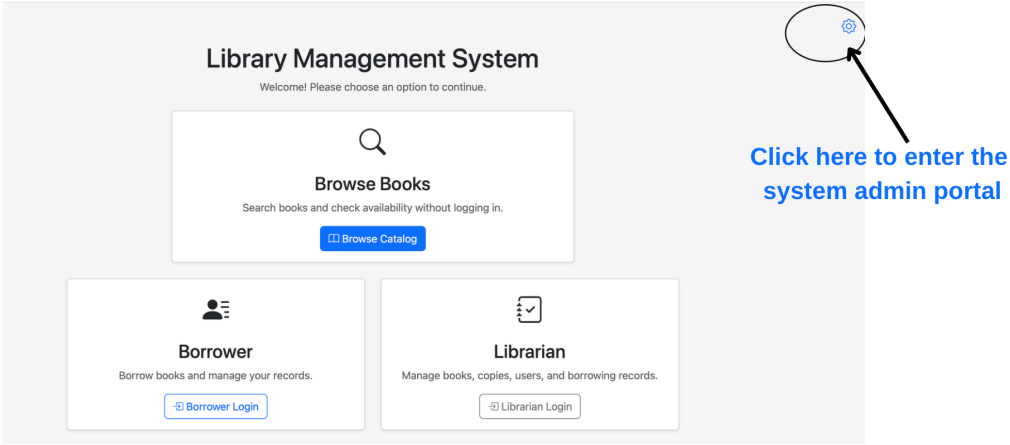
The combination of unit tests, integration tests, extensive edge-case coverage, and targeted bug fixes demonstrates that the system validates current functionality and contributes substantially to the system's long-term scalability, reliability, and robustness.

Note for source code

This is a system that targets librarians and borrowers. However, the librarian can't register. Instead, the system admin can add the librarian's information to the database.

The following is the login information for the system administrator:

admin@library.com
Admin123!



You can use the librarian information below directly, or you can add a new librarian in the system admin page.

ray@admin.com

admin123